

Gnani Speech-to-Text — Offline Engineering Reference

****Source:**** Official Gnani documentation
****Primary page:**** <https://docs.gnani.ai/api/STT/speech-to-text>
****Related Batch documentation:**** <https://docs.gnani.ai/api/STTBatch/Introduction>
****Related Create Job documentation:**** https://docs.gnani.ai/api/STTBatch/Create_Job
****Retrieved:**** 2026-10-01

> This file is an offline engineering reference assembled from the official Gnani documentation pages. It is intended to be placed in the Antigravity workspace so the agent can consult the local reference during implementation. The live official documentation remains the authoritative source if the provider changes.

1. REST Speech-to-Text

Purpose

Gnani's REST STT endpoint performs a single synchronous HTTP transcription request and is intended for short prerecorded audio clips.

The documentation describes:

- Short clips up to 60 seconds → REST
- Live microphone / streaming → Realtime
- Large files or bulk jobs → Batch STT

Endpoint

POST <https://api.vachana.ai/stt/v3>
Content-Type: multipart/form-data

Authentication

Header:

X-API-Key-ID: <api-key>

The key is a Gnani Prisma v2.5 API key obtained from the Gnani APIs dashboard.

REST request

All request parameters are multipart form fields.

Required:

audio_file
language_code

Supported REST audio formats:

WAV
MP3
OGG
FLAC
AAC
M4A

REST duration:

Maximum: 60 seconds
Ideal: 30 seconds or less

REST language codes

The REST API documents these 10 Indian language codes:

bn-IN Bengali
en-IN English
gu-IN Gujarati
hi-IN Hindi
kn-IN Kannada
ml-IN Malayalam
mr-IN Marathi
pa-IN Punjabi
ta-IN Tamil
te-IN Telugu

REST output format

format:

verbatim default; raw spoken-form output
transcribe enables inverse text normalization (ITN)

With format=transcribe, Gnani normalizes:

- numbers
- currency
- dates
- times
- phone numbers

Example:

five thousand rupees
→
₹5,000

Native numerals

itn_native_numerals=true makes ITN use the target language's native digit script where applicable.

Example for Hindi:

₹5,000
→
₹५,०००

Substitution

enable_substitution=true enables post-transcription find-and-replace using substitution_map.

Rules:

- maximum 10 rules
- case-insensitive
- Unicode word boundaries
- longer phrases take priority
- single-pass, no cascading

Example:

```
[
  {
    "patterns": ["okay", "ok", "alright"],
    "replacement": "acknowledged"
  },
  {
    "patterns": ["rupees", "rs"],
    "replacement": "INR"
  }
]
```

The documentation states that substitution affects the final client-visible transcript while billing/analytics retain the original raw transcript.

Word boosting

Fields:

```
bias_list
bias_score
```

bias_list:

- up to 100 words
- single words only
- letters only
- no spaces/digits/punctuation/symbols

Allowed bias_score:

```
0.0
0.5
1.0
1.5
2.0
```

Boosting is not available on Batch STT.

REST success response

Example:

```
{
  "success": true,
  "request_id": "req_abc123",
  "timestamp": "20251226_143052.123",
  "transcript": "□□□□□□, □□ □□□□ □□□?"
}
```

Fields:

```
success
request_id
timestamp
transcript
```

REST errors

Documented examples include:

```
400 invalid request / unsupported audio / duration over 60 seconds
403 forbidden / API key / organization / credits
429 rate limit
500 internal API error
503 service unavailable
```

The documentation recommends retrying transient 500/503 errors with backoff.

2. Python SDK

The official documentation lists:

```
pip install gnani-vachana
```

Requires Python 3.10+.

Example:

```
from gnani.stt import GnaniSTTClient
```

```
client = GnaniSTTClient(api_key="your-api-key")
result = client.transcribe("recording.wav", language_code="hi-IN")
print(result["transcript"])
```

The SDK documentation states that it handles multipart construction, authentication headers, and retries automatically.

For this take-home, the implementation should still isolate the provider behind an application-level interface so that vendor-specific behavior does not leak throughout the codebase.

3. Important REST-vs-Batch Design Decision

The assignment targets long audio.

The REST endpoint is NOT suitable for the target because the official REST documentation caps audio at 60 seconds.

The official Batch API is the intended API for long files.

Therefore:

short $\leq$ 60 seconds
→ REST is possible

long audio
→ Batch STT

For this project, Batch STT should be the primary transcription path.

4. Batch STT

Official page:

<https://docs.gnani.ai/api/STTBatch/Introduction>

Batch STT performs asynchronous transcription of long or multiple audio files.

Flow:

```
CREATE
POST /stt/v3/batch/jobs
↓
START
POST /stt/v3/batch/jobs/{job_id}/start
↓
POLL
GET /stt/v3/batch/jobs/{job_id}
↓
FILES
GET /stt/v3/batch/jobs/{job_id}/files
↓
DOWNLOAD
GET transcript_url
```

Important:

> Creating a job does NOT start transcription. /start must be called.

Batch statuses

CREATED
STARTING
QUEUED
IN_PROGRESS
COMPLETED
PARTIAL_FAILURE
FAILED
START_FAILED
CANCELLING
CANCELLED

Terminal statuses:

COMPLETED
PARTIAL_FAILURE
FAILED
START_FAILED
CANCELLED

Polling

The documentation recommends:

minimum poll interval: 10 seconds
prefer ~30 seconds for 100+ files

Polling more frequently risks rate limits.

5. Batch Create Job

Endpoint:

POST <https://api.vachana.ai/stt/v3/batch/jobs>

Authentication:

X-API-Key-ID: <api-key>

For a cloud-storage source, the request uses:

Content-Type: application/json

Example:

```
{
  "config": {
    "model": "gnani-prisma-v2.5",
    "language_code": "en-IN",
    "mode": "transcribe",
    "with_diarization": false,
    "is_multi_channel": false,
    "with_denoise": false
  },
  "source": {
    "type": "cloud_storage",
    "auth": {
      "mode": "public"
    },
    "paths": [
      "https://cdn.example.com/audio/call-001.mp3"
    ]
  }
}
```

Response:

```
{
  "job_id": "...",
  "status": "CREATED",
  "created_at": "...",
  "message": "Job created. Call POST /stt/v3/batch/jobs/{job_id}/start to begin processing.",
  "total_files_accepted": null
}
```

6. Cloud Storage / Presigned URL

This is particularly important for the Audio Notes Platform.

The official Create Job documentation states:

- Public HTTPS URLs are supported.
- S3 presigned URLs are supported.
- Authenticated private-bucket access is not supported.
- URLs are validated at Start Job, not Create Job.

Therefore the architecture can keep R2 private while giving Gnani a short-lived presigned HTTPS URL for the specific object.

Conceptual flow:

```
Browser
↓
direct multipart upload
↓
private Cloudflare R2 object
↓
```

FastAPI creates short-lived presigned GET URL
↓
Gnani Batch Create Job
↓
Gnani fetches object

Do NOT make the entire R2 bucket public.

7. Batch Limits

Every file:

Maximum audio duration: 4 hours
Minimum audio duration: 0.1 seconds
Maximum speech segments detected: 5,000 per file

A file longer than 4 hours is rejected with:

AUDIO_TOO_LONG

Direct multipart upload to Gnani

Maximum files/job: 100
Maximum size/file: 10 MB
Maximum ZIP compressed size: 50 MB
Maximum ZIP decompressed size: 200 MB
Maximum ZIP compression ratio: 50:1

Cloud storage source

Maximum size/file: no byte limit
Maximum download time/file: 30 minutes
Maximum files/job: not currently capped

This distinction is critical.

The 10 MB restriction is for uploading files directly to Gnani.

A cloud-storage source is bounded by the 4-hour duration cap and the 30-minute download-time limit rather than a byte-size cap.

8. Consequence for This Assignment

The project's target is:

maximum duration: 300 minutes
maximum file size: 5 GB

300 minutes = 5 hours.

Gnani Batch supports only 4 hours per file.

Therefore the architecture MUST NOT claim that one 300-minute file can always be sent as a single Gnani Batch file.

Instead:

≤ 4 hours
→ one Gnani Batch file

> 4 hours and ≤ 5 hours
→ FFmpeg chunking fallback

The chunking fallback is therefore actually required by the project's 300-minute target.

Recommended strategy:

source audio
↓
FFmpeg split into chunks < 4 hours
↓
each chunk → Gnani Batch

```

↓
collect transcripts
↓
restore timestamp offsets
↓
merge ordered segments
↓
single logical transcript

```

Use conservative chunk sizes rather than splitting exactly at the 4-hour boundary.

9. Batch Configuration

Documented configuration:

```

model          required
language_code  required
mode           default transcribe
with_diarization default false
num_speakers   required when diarization=true; max 2
is_multi_channel default false
with_denoise    default false

```

Batch documentation states that ITN is NOT supported for STT Batch.

Therefore do not design the Batch path around REST's format=transcribe / ITN features.

10. Batch Supported Languages

Batch documents:

```

bn-IN Bengali
en-IN English
hi-IN Hindi
kn-IN Kannada
ml-IN Malayalam
mr-IN Marathi
ta-IN Tamil
te-IN Telugu

```

Important:

```

gu-IN Gujarati
pa-IN Punjabi

```

are supported on REST/Realtime but NOT on Batch STT.

If the application must support Gujarati/Punjabi for files longer than the REST limit, a provider-compatible strategy or chunked REST/Realtime strategy must be considered. Do not silently claim Batch supports them.

11. Language Identification

Batch allows:

- one language code, OR
- up to three comma-separated language codes

Example:

```
hi-IN,en-IN
```

Important limitation:

> Language identification is per file, not per segment.

The documentation says the file is transcribed entirely in one resolved language.

Therefore this is NOT the same as arbitrary intra-file Hindi/English code-switching.

The project should describe this accurately.

Recommended UX:

Auto-detect
OR
specific language
OR
up to 3 candidate languages

only where supported by the selected Batch path.

Do not promise true segment-level code-switching if the provider does not provide it.

12. Batch Transcript Result

The transcript can be obtained from transcript_url.

Example:

```
{
  "full_transcript": "hello how can i help you today ...",
  "model": "gnani-prisma-v2.5",
  "language_code": "en-IN",
  "duration_seconds": 6.01
}
```

The webhook sample additionally shows segment-level data:

```
{
  "segment_id": 0,
  "start_time": 0.0,
  "end_time": 0.71,
  "text": "hello",
  "speaker_id": 1,
  "confidence": null,
  "language_detected": null,
  "sentiment": null,
  "emotion": null
}
```

and:

```
full_transcript
created_at
model
language_code
duration_seconds
mode
segments
```

The application should preserve the complete raw response.

It should normalize only the fields needed for:

- display
- search
- timestamp seeking

13. Transcript URL Expiration

The Batch documentation states:

transcript_url expiry: 1 hour

Therefore:

- download transcript promptly after completion
- do not store the provider's temporary URL as the permanent transcript location
- store the downloaded raw JSON in PostgreSQL/object storage as appropriate

14. Webhooks

Batch supports:

`callback_url`

Events:

`job.completed`
`job.partial_failure`
`job.failed`
`job.cancelled`

The documentation recommends treating webhooks as best-effort and says polling remains the source of truth.

For this take-home, a polling-based worker is simpler.

Preferred project flow:

```
ARQ job
↓
create Gnani Batch job
↓
start Gnani job
↓
poll ≥10s
↓
completed
↓
get transcript_url
↓
download transcript
```

A webhook can be added later if desired.

15. Denoising

`with_denoise=true` is available.

The documentation recommends it for noisy audio and leaving it off for clean audio.

It adds processing time.

For this take-home:

`default: false`

unless the product later adds an explicit user option.

16. Diarization

Batch supports:

`with_diarization=true`
`num_speakers <= 2`

This assignment does not require speaker labeling.

Do not add speaker UI unless it provides a clear benefit without increasing scope.

The raw provider response can still be preserved.

17. Common Batch Mistakes

Do not:

- forget /start
- expect full_transcript from status endpoint
- use /v1/jobs/... based on the response message type
- upload silence/tones and expect meaningful transcription
- give Gnani an unreachable URL
- poll every 1-2 seconds
- use denoise on clean audio unnecessarily
- keep using an expired transcript_url

Correct flow:

```

create
→ start
→ poll
→ files
→ download transcript

```

18. Recommended Project Integration

For the Audio Notes Platform:

```

Browser
↓
R2 multipart upload
↓
FastAPI verifies upload
↓
ARQ process_transcription job
↓
If duration ≤ 4h:
  Gnani Batch using R2 presigned URL

```

If duration > 4h:

```

FFmpeg chunking
↓
Gnani Batch per chunk
↓
merge segments/transcripts
↓
store raw Gnani response
↓
normalize transcript
↓
store transcript segments
↓
ARQ summary job
↓
Gemini Flash
↓
store summary
↓
COMPLETED

```

19. Official Source URLs

REST:

<https://docs.gnani.ai/api/STT/speech-to-text>

Batch introduction:

<https://docs.gnani.ai/api/STTBatch/Introduction>

Batch create job:

https://docs.gnani.ai/api/STTBatch/Create_Job

Gnani ASR playground referenced by the assignment:

<https://app.gnani.ai/voice/speech-to-text>

20. Verification Note

This offline reference was assembled from the official Gnani documentation available on 2026-10-01.

The official Batch page states it was last verified on 2026-08-05 and describes a live-verified Create → Start → Poll → Files → Transcript workflow.

If the live documentation changes, update this offline reference and the project ADRs before changing implementation behavior.